Ukrainian Catholic University

Bachelor Thesis

Autonomous Ground Navigation and Dynamic Obstacle Avoidance for Husky Robot in Pedestrian Areas

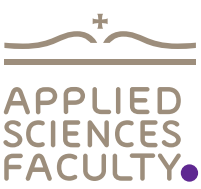
Author:
Radomyr Husiev

Supervisors:
Oleg Farenyuk
Oleksandr Kosovan

A thesis submitted in fulfillment of the requirements for the degree of Bachelor of Science

in the

Department of Computer Sciences and Information Technologies
Faculty of Applied Sciences



Lviv 2026

UKRAINIAN CATHOLIC UNIVERSITY

Faculty of Applied Sciences

Bachelor of Science

Autonomous Ground Navigation and Dynamic Obstacle Avoidance for Husky Robot in Pedestrian Areas

by Radomyr Husiev

Abstract

TODO

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Background and Motivation

Ground vehicles – very important field; especially for tedious, far-away, or dangerous tasks.

It is often infeasible to manually control such vehicles due to their sheer number and task volume. Physical barriers like distance or signal interference mean manual control can fail. Therefore, autonomous systems are a necessity for the robot to complete its task and navigate to desired locations independently.

Outdoors, there are challenges of an unpredictable environment: unexpected/moving obstacles, surrounding agents (like other vehicles or pedestrians) do not cooperate, uneven terrain, etc.

Today, there are ready-made solutions for autonomous navigation (like default Nav2 in ROS). However, modern outdoor systems often rely on heavy sensors (3D LiDAR, RGB-D cameras) and algorithms to process this data (Visual SLAM, MPPI). These require high computational resources, lots of RAM, and sometimes GPUs. This is expensive, it drains battery and not always possible on simple low-cost robots.

Therefore, there is a motivation to see what can be accomplished using a minimal, lightweight sensor suite. By avoiding heavy computations, we can create a more accessible and energy-efficient system.

Instead of calculating a map from scratch using SLAM, we can use existing, semantic data like OpenStreetMap (OSM) for global routing. Instead of 3D point clouds or image processing, we can use a simple 2D LiDAR to handle immediate, dynamic obstacles (like pedestrians or trees). Instead of visual odometry, we can rely on standard GPS fused with an IMU and wheel odometry.

Together, these lightweight components theoretically provide all the necessary data for a robot to navigate (albeit in lower fidelity and density, higher sparcity). This thesis aims to prove that such a minimal setup is actually viable in the real world.

The setup includes a ground differentially driven robot outdoors in a park. Pedestrian paths provide a semi-structured topologically known environment. Unlike road networks, pedestrian zones lack traffic rules, lane changes, or traffic lights, eliminating the need for heavy semantic computer vision processing. However, unlike completely unstructured off-road scenarios, these paths have clear geometric boundaries that can be modeled and tracked. The system's objective is to navigate autonomously from a start point to a target point without human intervention.

1.2 Objectives

- Path planning – find a global route (using graph search algorithms like A*) based on environment map (OpenStreetMap XML data).

- Obstacle detection – process 2D LiDAR data to find obstacles.
- Local navigation – combine spatial localization (GPS and IMU+Wheel Odometry – sensor fusion via Extended Kalman Filter) and local path planning (using an optimization algorithm Timed Elastic Band) to avoid obstacles and follow the global path.

Conducted real-world experiments in public environment (Stryyskyi Park). Also in simulation – more times here. Evaluated the system's success rate, safety, and travel time. Provided a baseline comparison against the standard Nav2 implementation.

1.3 Structure Of The Thesis

Chapter 2

Related Work

2.1 Path and Trajectory Planning

The problem of autonomous navigation classically divided into:

- Global path planning
- Local trajectory planning/obstacle avoidance

Global – graph-based approaches like Dijkstra's or A*, because reliable and optimal on static maps.

But when moving in dynamic environments, real-time local planning is required. Early approaches, such as the Artificial Potential Fields (APF) method, modeled the environment as a physical field: the goal attracts, the obstacles repel. But it suffered from local minimums and worked worse with tight gaps. Later, the Vector Field Histogram (VFH) was introduced: simpler and with better performance, though it struggled with complex paths and kinematic constraints of vehicles.

There is a widely used Dynamic Window Approach (DWA), which takes a set of feasible new velocities for the next few seconds, checks each one for obstacles, goal etc. → chooses the best. The resulting trajectory is addresses the kinematics (smooth geometry of the path), but it struggles to produce paths for complex environments. Therefore, it is included in ROS NAV2 stack, but as a simpler and less powerful planner.

Then there is Elastic Band planner, which tries to predict the whole path as a band between two points, which stretches to avoid obstacles. But it does not produce a trajectory, it only provides with a geometric path (instead of telling the controller what speeds to drive at what points in time, it only provides a path – a sequence of points the robot should follow).

Timed Elastic Band (TEB) is a continuation, which shifts toward a Model Predictive Control (MPC)-style paradigm. Unlike purely reactive methods, MPC-style algorithms use a receding-horizon approach: they predict and optimize a continuous trajectory over a future time window while enforcing kinodynamic constraints (the geometrical and physical limitations, such as maximum acceleration and turning radius). TEB optimizes this local trajectory for execution time and separation distance from obstacles. More recently, sampling-based MPC algorithms like Model Predictive Path Integral Control (MPPI) have emerged for aggressive, high-speed driving. NAV2 once used TEB for its advanced trajectory planner, but recently moved to MPPI.

While MPC methods generate superior trajectories compared to reactive methods like DWA, they require higher computational overhead. MPPI in particular requires parallel sampling that often necessitates a GPU, making it unsuitable for constrained systems. Therefore, a CPU-optimized MPC approach like TEB provides a middle ground between performance and efficiency.

2.2 Localization

You need to know the location of the robot to know where to drive. Currently one of the most used methods is Simultaneous Localization and Mapping (SLAM), particularly Visual-Inertial SLAM (VI-SLAM) and 3D LiDAR-based SLAM. However, these methods require lots of computational power. For example [Schmidt] demonstrated that VI-SLAM algorithms can use 4-6 GB of RAM for the mapping node and their accuracy degrades significantly if the frame rate is lowered to save processing power. Furthermore, outdoor environments – specifically parks with heavy vegetation – cause SLAM algorithms to drift or lose tracking entirely due to dynamic foliage and lack of distinct structural features.

Because of these limitations, using Extended Kalman Filter (EKF) to fuse GPS, Inertial Measurement Units (IMU) and wheel odometry can be used as a stable alternative. [Moore] showed that while wheel odometry drifts immensely over time, fusing it with IMU provides an improvement. Adding GPS, even if jumpy and unpredictable under trees, further reduces the long-term drift, providing a reliable localization without the overhead of SLAM.

2.3 Semantic Mapping

SLAM is not only for localization, but also for mapping. To further avoid the costs of SLAM, researches explored using existing semantic maps for global planning. [Chen] note a growing trend of reducing SLAM dependency by navigating via pre-existing semantic data.

OpenStreetMap (OSM) is a crowdsourced geographic database with structured tags describing road types, surface materials, access restrictions, and geometry. [Hentschel] showed that autonomous navigation can be successfully performed using OSM geodata. There is a distinct benefit to using OSM over SLAM: it provides the semantic context of the environment. Instead of viewing the world as a grid of occupied or free space, OSM distinguishes paths from grass and describes surface materials. Furthermore, [Stahr] utilize OSM attributes such as path width to create boundaries for local planners, which limits the search space for trajectory planning algorithms like TEB.

2.4 Obstacle Abstraction

Even with a lightweight mapping approach, processing raw sensor data for immediate obstacle avoidance remains resource-intensive. Compared to 2D LiDAR, 3D one has much more points (a vertical dimension). With 3D LiDAR there also emerges a problem of ground points, which need to be filtered out in order not to consider the roal itself as an obstacle. Due to these limitations 3D LiDAR becomes computationally expensive.

Using 2D LiDAR solves the vertical complexity but still produces hundreds of data points per scan. To optimize this, researchers use algorithms to extract geometric primitives out of point clouds. [Catapang] and [Nguyen] walked through the scan and grouped all points at the distance lower than the vehicle diameter into clusters. Then to extract the clusters into geometric primitives [Sezer] used circle approximation. [Nguyen] compared several algorithms and found that the "Split and Merge" approach is the most efficient and very accurate for converting raw LiDAR points into lines.

While some approaches such as [Du] treat dynamic agents (like pedestrians) differently from static obstacles, implementations often rely on Reinforcement Learning

and 3D tracking, which conflicts with computational efficiency. However, there are approaches like Arras et al. [2] that demonstrated that detecting dynamic agents like pedestrians in 2D range data can be achieved using static geometric traits alone.

To further optimize the process, efficient feature detection is required. Xavier et al. [14] introduced an approach with an $O(n)$ linear complexity that utilizes geometric pre-filtering and Inscribed Angle Variance (IAV). This allows a system to accurately categorize features into straight lines, arcs (trees), and legs within milliseconds.

2.5 Conclusion

Chapter 3

Theoretical Background

3.1 Sensor Characteristics

Autonomous navigation systems rely on a number of sensors to perceive the environment and estimate the vehicle's state. Because a core objective of this research is resource efficiency, it is important to understand the capabilities and limitations of the minimal sensor suite:

- **Wheel Odometry:** Estimates the robot's relative displacement by measuring the rotation of its wheels. While useful for short-term tracking, it accumulates drift over time due to mechanical slip, which is especially prominent in skid-steer UGV configurations.
- **Inertial Measurement Unit (IMU):** Contains accelerometers and gyroscopes to measure linear acceleration and angular velocity. It provides frequent motion updates but suffers from integration drift and bias, meaning small noise errors grow quadratically over time.
- **Global Positioning System (GPS):** Provides absolute geographic positioning (latitude, longitude) by triangulating satellite signals. However, it is prone to high-frequency noise, low update rates, and signal degradation under trees or near tall structures.
- **2D LiDAR (Light Detection and Ranging):** Uses laser pulses to measure distances in a single horizontal plane, outputting a point cloud in polar coordinates. While computationally much lighter than 3D LiDAR, it lacks vertical resolution and cannot distinguishing between ground features (such as hills) and vertical obstacles.

3.2 Differential Drive UGV

In this thesis, the robot uses a four-wheeled skid-steer differential drive configuration. While the machine has four points of contact with the ground, steering is achieved through a velocity differential between the left and right wheel pairs.

To generate realistic and executable trajectories, autonomous planners must respect the physical limitations of the robot, which are broadly categorized into kinematic and dynamic constraints. Together, these form the robot's kinodynamic constraints.

- **Kinematic constraints**. A differential drive system acts under non-holonomic kinematic constraints: it can rotate in place and move forward or backward, but it cannot move laterally (it cannot slide sideways). Therefore, the robot's

state transitions are controlled by its linear (v) and angular (ω) velocities [7]. If a planner generates a path that requires lateral movement, the hardware physically cannot execute it.

- **Dynamic constraints**. The UGV has physical mass and limited motor torque, so it cannot change velocities instantly. Therefore, dynamic cosntraints define robot's maximum achievable velocities (v_{max}, ω_{max}) and maximum accelerations (a_{max}, ϵ_{max}).

Unlike two-wheeled systems, the four-wheeled configuration introduces lateral slip (skidding) during rotation, which increases drift in wheel-based odometry [6]. Thus, accurate sensor fusion becomes more important to correct for the drift.

3.3 Hierarchical Navigation Architecture

Classical autonomous navigation typically employs a hierarchical, two-level architecture. Planning a kinematically feasible, collision-free trajectory from a starting point to a distant goal in a single step is computationally too expensive. Therefore, the problem is split into global and local planning. The global planner operates on a static map to find a purely topological route to the destination, ignoring the vehicle's kinematics and dynamic obstacles. The local planner then acts upon a small sliding window of this global route, functioning in real-time and generating motion commands that avoid immediate, unexpected obstacles.

3.4 Spatial Representation

3.4.1 Coordinate Systems

GPS sensors output data in geodetic coordinates (latitude, longitude, altitude). Trajectory planners operate in Cartesian coordinates. Geodetic coordinates must be projected onto a local tangent plane in order for local planner to calculate distances in meters.

3.4.2 Graph-Based Map Representation

For a global planner to generate paths, it requires the environment to be abstracted into a directed semantic graph [5]. Unlike grid-based maps that treat the world as a matrix of occupied or free pixels, a semantic graph represents vertices as discrete spatial coordinates (such as path intersections), and edges as traversable paths (such as pedestrian roads). Complex paths made of multiple physical segments are simplified into single edges connecting these intersections. The weight of each edge is the sum of all its original segment lengths. This preserves distance accuracy while reducing the number of vertices the planning algorithm must evaluate.

The A* algorithm [4] navigates this graph. It calculates the shortest path by minimizing the function $f(n) = g(n) + h(n)$, where $g(n)$ is the exact cost from the start vertex to vertex n and $h(n)$ is the estimated heuristic cost. For A* to guarantee the mathematically shortest path, the heuristic must be admissible, meaning it must never overestimate the actual cost to the goal [10]. Because a straight line is always the shortest possible distance between two points, the Euclidean heuristic is admissible.

With the heuristic cost in place, the algorithm first processes paths closer to the goal, thus the frontier of the search stretches in an elliptical shaped, pointing at the

target. This naturally reduces the number of vertices processed, while preserving accuracy.

3.5 Trajectory Optimization

The Timed Elastic Band (TEB) approach [12] formulates local navigation as a multi-objective optimization problem with a receding-horizon strategy. At each control step, the algorithm predicts a short-term trajectory over a finite time window, applies the first generated control command, and then continuously re-optimizes the trajectory as new sensor data arrives. The trajectory itself is defined as a sequence of intermediate poses between the start and goal. Each pose consists of x, y coordinates (in meters), θ (heading angle, in radians) and Δt (time interval, in seconds). The trajectory is initially generated as a sequence of points, with Δt and θ linearly interpolated.

The trajectory is then deformed based on constraints and obstacles. The problem is framed as a non-linear least squares problem to be optimized. The robot's poses act as the parameters. The physical and spatial constraints act as cost functions (also known as residuals). The optimization engine minimizes a weighted sum of these cost functions. In a least squares solver, the total cost is defined as $\frac{1}{2}\sum R_i^2$. Therefore, to apply a weight w to a cost, the raw error is multiplied by $\sqrt{w}$, ensuring the squared residual provides the strictly weighted cost.

The standard formulations of these residuals [12] ensure safety, kinematic feasibility, and efficiency:

3.5.1 Obstacle Cost

To prevent collisions, the solver penalizes poses that fall within the safety radius of geometric primitives. For a circular obstacle, the cost calculates the shortest distance from the obstacle center O to the robot line segment AB. P is the closest point on AB and is represented using $P = A + t(B - A)$. The projection scalar t is clamped to $[0, 1]$ to ensure the closest point lies strictly on the segment. Let $d = \|O - P\|$:

$$R_{obs} = \begin{cases} \sqrt{w} \cdot (r_{obs} + r_{safe} - d) & \text{if } (r_{obs} + r_{safe}) > d \\ 0 & \text{otherwise} \end{cases} \tag{3.1}$$

3.5.2 Kinematic Cost

To enforce the differential-drive constraint, the robot's motion between two configurations is modeled as an arc segment of constant curvature [12]. This requires the angle ϑ_i between the robot's heading and the displacement vector $\mathbf{d}_{i,i+1}$ at the start configuration to be equal to the corresponding angle ϑ_{i+1} at the final configuration. Rather than optimizing these angles directly, the constraint is formulated as a cross product:

$$\begin{pmatrix} \cos\theta_i \\ \sin\theta_i \\ 0 \end{pmatrix} \times \mathbf{d}_{i,i+1} = \mathbf{d}_{i,i+1} \times \begin{pmatrix} \cos\theta_{i+1} \\ \sin\theta_{i+1} \\ 0 \end{pmatrix} \tag{3.2}$$

Simplified, the constraint reduces to a single scalar equation, yielding the resulting least-squares residual R_{kin}:

$$R_{kin} = \sqrt{w} \cdot \Big[(\cos\theta_i + \cos\theta_{i+1})\Delta y - (\sin\theta_i + \sin\theta_{i+1})\Delta x\Big] \tag{3.3}$$

3.5.3 Dynamics Costs

This cost imposes penalties if the physical limits of a UGV are exceeded. Using finite differences, the linear velocity cost is formulated as:

$$R_v = \begin{cases} \sqrt{w} \cdot (|v| - v_{max}) & \text{if } |v| > v_{max} \\ 0 & \text{otherwise} \end{cases} \tag{3.4}$$

Analogous formulations are applied for linear acceleration, as well as angular acceleration and velocity, ensuring angle differences are normalized to $[-\pi, \pi]$. Two poses are required to calculate the finite differences for the velocity costs. Three poses are needed for acceleration costs, as they require two consecutive velocities to calculate the change in velocity.

3.5.4 Time Cost

To prevent the solver from minimizing other penalties by outputting infinite execution times, a penalty is applied directly to the Δt parameters. This cost forces the system to progress toward the goal:

$$R_t = \sqrt{w} \cdot \Delta t \tag{3.5}$$

3.5.5 Solvers and Jacobians

Traditionally, the TEB algorithm models the trajectory optimization as a hyper-graph, where robot poses represent the nodes and the kinematic or spatial constraints represent the edges. However, this hyper-graph formulation is mathematically equivalent to a standard non-linear least-squares optimization problem. In a least-squares framework, the poses act as the parameter blocks, and the constraints act as the residual blocks.

To solve this objective function, optimization engines require Jacobians—matrices of partial derivatives indicating how changes in the trajectory parameters affect the total cost. These matrices are used to find the gradient of the cost functions and can be calculated analytically or numerically. Because trajectory constraints usually affect only adjacent poses, the resulting mathematical system is highly sparse. Solvers apply sparse factorization methods (such as Cholesky decomposition) to efficiently solve the equations, iteratively updating the parameters until they converge on a smooth, physically feasible trajectory.

3.5.6 Temporal Coherence

If an optimizer starts from a blank slate (a cold start) at every control step, it may fail to converge within strict time limits and may lead to inconsistent trajectories. Model Predictive Control (MPC) approaches, including TEB, solve this by utilizing a Warm Start [11]. Because the environment changes slightly between control frames, the optimized trajectory from the previous time step can be preserved and used as the initial guess for the next iteration. This temporal coherence reduces the number of iterations required for the solver to find a local minimum.

3.6 Sensor Fusion

Sensors contain noise and inherent inaccuracies. Wheel odometry accumulates drift from mechanical slippage, while IMU exhibits integration drift. Small offsets in IMU

sensor bias and noise result in quadratic position error growth and orientation errors caused by gravity leakage [8]. Global Positioning System (GPS) sensors provide absolute global coordinates but suffer from high-frequency noise and interference under tree cover or near structures [13].

The Kalman Filter fuses sensor streams into a single reliable state estimate [8]. It maintains a state vector and a covariance matrix, which represents the system's current uncertainty for each sensor. It uses IMU and wheel odometry to continuously predict the robot's next state, but with time uncertainty grows. GPS is used to correct the predicted state upon measurement and reduce uncertainty.

The standard Kalman Filter assumes linear system dynamics. However, mobile robots involve non-linear movements, such as turning and circular trajectories, which require trigonometric functions. The Extended Kalman Filter addresses this by linearizing motion and models at the current state estimate (achieved through Taylor series expansion) [8].

This fusion also ensures that even if a sensor like GPS becomes temporarily unavailable, the robot can continue to navigate using its internal motion model until the absolute signal is restored.

3.7 Geometric Feature Extraction

2D LiDAR outputs data in polar coordinates (distance and angle). To be used by the optimization engine, this point cloud must be segmented and abstracted into geometric primitives (lines and circles) in Cartesian space.

3.7.1 Pre-processing and Clustering

Raw LiDAR scans frequently contain isolated noise spikes and artifacts. Therefore, the points are first passed through a median filter to reject extreme outliers. The filtered point clouds are then grouped into discrete clusters using distance-based segmentation [9]. Points are assigned to a single sequential object if the Euclidean distance between consecutive measurements P_i and P_{i+1} is below a defined break distance. The vehicle diameter plus some padding can be used as an effective threshold, as narrower gaps would not be passable by the robot.

3.7.2 Shape Classification

Clusters must be categorized as linear or circular to apply the correct geometric extraction logic. For clusters with three or more points, a geometric pre-filter [14] is used to evaluate the curvature. A chord vector is drawn between the cluster's endpoints, P_1 and P_3, and the cross product is used to calculate the perpendicular arc depth to the median point, P_2. If this depth is bounded between 10% and 70% of the chord length, the cluster exhibits sufficient convexity to be classified as a circle. Clusters falling outside this range, or clusters exhibiting an excessive maximum internal point-to-point distance, are classified as lines.

3.7.3 Line Fitting

Line extraction uses the Split-and-Merge algorithm [9]. First, a chord is drawn between the cluster's endpoints. The algorithm calculates the perpendicular distance from every internal point to the chord. If the maximum distance exceeds a predefined threshold, the cluster is split at that peak point into two sub-scans, and the process

recurses. The result is a set of distinct linear segments that approximate complex shapes like walls or curbs.

3.7.4 Circle Fitting

For clusters identified as circular obstacles (such as trees or pedestrians), geometric parameters (center and radius) must be extracted. Algebraic methods, such as the circumcenter approach [14], attempt to find an analytical solution by selecting three points and computing the intersection of their perpendicular bisectors.

While mathematically exact for perfect curves, algebraic fitting frequently fails when applied to 2D LiDAR data. Because LiDAR only senses the front face of an obstacle, the returned points form a shallow arc. For distant or small obstacles, the limited resolution of the scanner causes these arcs to approach a straight line. Furthermore, because the laser beam has a physical width, reflections from the highly angled flanks of cylindrical objects register prematurely. This smearing effect flattens the perceived curve, causing traditional algebraic methods to systematically overestimate the cylinder's diameter by up to 19% [3].

Chapter 4

Proposed Solution

4.1 Problem Formulation

The objective of this research is to enable an unmanned ground vehicle (UGV) to reach a target position (defined via GPS coordinates or a relative Cartesian offset) from an initial starting point entirely without human intervention. The solution presents an OSM-based hierarchical autonomous navigation system operating in a semi-structured outdoor pedestrian environment. A semi-structured environment, such as a park, provides fixed paved paths but introduces dynamic, unpredictable obstacles like pedestrians.

The core constraint of the problem is to design a resource-efficient setup using a minimal sensor suite (2D LiDAR, GPS, IMU, wheel odometry) without relying on computationally intensive and energy-demanding hardware like GPUs, 3D LiDARs, or RGB-D cameras. By combining semantic-graph-based global routing with receding-horizon local trajectory optimization, the system minimizes computational demands while maintaining reliable obstacle avoidance and spatial accuracy.

4.2 System Architecture

The system consists of two ROS2 nodes:

- **Controller Node**: Executed directly on the Unmanned Ground Vehicle (UGV). It processes raw sensor data, extracts geometric primitives for obstacle abstraction, and computes both the global path and local trajectory. Running this node onboard minimizes network latency and ensures real-time responsiveness.
- **Visualization Node**: Operates off-board on a separate machine within the same local network. It subscribes to the Controller Node to render obstacles, paths, and trajectories, displaying them in a UI. It also allows entering the desired goal location or canceling navigation.

4.3 Software Implementation

4.3.1 Semantic Mapping and Global Routing

OSM is used for a semantic map. To process this data, the Python `osmnx` library is utilized. It allows for the extraction of a multigraph for a specific OSM polygon relation (such as the park boundary), where traversable paths act as edges and intersections act as vertices. To filter out non-navigable terrain, the following edge tags are evaluated:

- `surface=`. Surfaces other than `paved, asphalt, concrete, paving_stones, sett, cobblestone` are excluded.
- `highway=`. For example, `motorway` and `cycleway` are excluded.
- `access=` and `foot=`. If any of these is `no` or `private`, the road is discarded.

After initial filtering, the global path can be generated. The A* algorithm is used to get a sequence of vertices that need to be followed in order.

The local planner does not operate on a graph, instead it works in a Cartesian coordinate system. Therefore, the vertices sequence has to be converted into a sequence of 2D points connected by straight lines. Because physical footpaths are rarely perfectly straight, single edges in the graph are often represented in OSM as a sequence of smaller linear segments (stored within the `geometry` attribute). By splitting each edge into these underlying segments, the global planner outputs a sequence of 2D Cartesian waypoints suitable for the local planner.

4.3.2 Obstacle Abstraction

The local trajectory planner requires distinct geometric constraints rather than a dense point cloud. To achieve this, a custom perception pipeline is implemented to process 2D LiDAR scans into Cartesian geometric primitives based on the theoretical models described in Chapter 2.

Pre-processing and Clustering

Raw LiDAR scans are converted from polar to Cartesian coordinates. Points below a minimum range (rays reflected from the vehicle itself) are discarded. The scan is passed through a 1D median filter of size N to reject isolated noise. The points are then grouped sequentially, broken based on the break distance (the vehicle width plus a safety margin).

Shape Classification

Clusters with fewer than three points are automatically classified as circles. For larger clusters, if the maximum internal Euclidean distance exceeds the geometry split threshold, it is considered a line. For the remaining clusters, the Xavier heuristic [14] is applied. The arc depth is calculated, and if it falls between 10% and 70% of the chord length, the cluster is classified as a circle; otherwise, it is treated as a line.

Primitives Extraction

To ensure stability against sensor noise and bias introduced with algebraic curvature methods of circle extractino, a spatial heuristic is used. The maximum Euclidean distance between any two points in the cluster (the chord width) establishes an initial diameter estimate. To counteract sensor noise and the systematic cylinder overestimation identified by Forsman et al. [3], the radius (half diameter) is clamped between predefined physical minimum and maximum bounds. Instead of calculating a mathematical center from the arc, the visible middle point of the cluster is projected outward from the sensor origin by the estimated radius to approximate the true geometric centroid. Finally, the radius is expanded, so that all points in the cluster are enclosed. This method mitigates overestimations while implicitly providing a safety margin. Clusters classified as lines are processed using the recursive Split-and-Merge algorithm to yield distinct linear segments.

Virtual Boundaries

To restrict the UGV to the pedestrian path, virtual line obstacles are generated. While path width can be extracted from OSM tags, in practice these tags are often unmapped. Instead, the implementation uses a default value of 4 meters. Parallel line barriers are placed on both sides of the target trajectory, creating a safe routing corridor for the local planner.

4.3.3 Localization and State Estimation

An Extended Kalman Filter (EKF) is implemented via the `robot_localization` ROS2 package. The EKF is configured to fuse data from the internal IMU and wheel encoders for continuous state estimation. It is used for the trajectory updates, so as not to introduce sudden shifts for TEB local planner.

Another EKF is used to take into account low-frequency GPS data. It is projected onto a local Cartesian tangent plane and fused to reduce the drift. This second filter's output is used to update the direction toward the target, so that the vehicle finishes movement at the exact destination.

In fallback scenarios where GPS is disabled or unavailable, the direction to the global goal is only updated using the first filter, which uses IMU and wheel odometry.

4.3.4 Local Trajectory Optimization

A custom Python implementation of the Timed Elastic Band (TEB) algorithm was developed, utilizing the `pyceres` library for non-linear optimization.

Local Goal Selection and State Representation

Because optimizing a trajectory all the way to the final destination is infeasible to compute in real time, the TEB planner operates on a sliding window. At each control loop, the UGV's current position is projected onto the global path (the closest waypoint to the current position is selected). The algorithm traverses the path forward until an accumulated length is equal to the predefined lookahead distance. This coordinate becomes the temporary local goal for the optimizer.

To ensure the solver remains stable, the trajectory resolution is dynamically resized between iterations. The distances between consecutive poses are calculated. If a distance is lower than a lower bound, the pose is pruned, and the Δt of the previous one is increased by the Δt of the current one. If the distance is larger than the upper bound, a new pose gets inserted in the middle of the other two, and the Δt is halved and spread across the inserted pose and the first of the two old ones.

Custom Formulations of TEB Residuals

While standard kinematic, dynamic, time, and circular obstacle costs (as defined in Section 2.5) were utilized in the system, specific custom formulations had to be derived for linear obstacles and start conditions to maintain solver stability in Pyceres.

Line Obstacle Cost Calculating the exact shortest distance between two line segments introduces non-differentiable edge cases. To solve this and maintain smooth Jacobians for the Ceres solver, a custom residual was derived. The minimum distance from the trajectory segment (endpoints A, B) to the obstacle segment (endpoints C, D) is approximated using a LogSumExp (Softmin) function over the four

endpoint-to-segment distances ($d_A \dots d_D$):

$$d_{min} = -\frac{1}{\alpha} \ln \left(\sum_{k=1}^{4} \exp(-\alpha \cdot d_k) \right) \tag{4.1}$$

$$R_{line} = \begin{cases} \sqrt{w} \cdot (r_{safe} - d_{min}) & \text{if } r_{safe} > d_{min} \\ 0 & \text{otherwise} \end{cases} \tag{4.2}$$

Starting Dynamics Cost For the first segment of the trajectory (from the current position to the first planned pose), there is no "previous" pose to reference – and as such, there cannot be three poses to calculate acceleration. If unconstrained, the solver might generate a trajectory that begins at a high velocity while the physical robot is stationary, implicitly demanding an infinite, impossible instantaneous acceleration.

To prevent this, a custom dynamic cost is implemented specifically for the starting position. It uses the robot's current velocity ($\vec{v}_{curr}, \omega_{curr}$), retrieved directly from the state estimation sensors, to calculate the acceleration and angular acceleration costs for the first trajectory segment.

Jacobians and Ceres Problem Setup

The standard ROS implementation of TEB relies on C++ and the `g2o` library However, because the TEB objective function is fundamentally a sum of squared weighted residuals, it can be easily adapted into a general-purpose non-linear least-squares framework. For this custom Python architecture, Google's Ceres Solver [1] was chosen via the `pyceres` library.

While `g2o` uses predefined graph edges, Ceres allows for the manual definition of residual blocks. Because `pyceres` lacks the C++ compile-time automatic differentiation feature of the native Ceres library, the partial derivatives for all implemented residuals were calculated analytically and provided directly to the solver.

To ensure the optimization can run in real-time, an obstacle filter is applied before constructing the Ceres problem. Rather than evaluating every obstacle against every pose, the system pre-calculates distances and only adds residual blocks to the solver for obstacles posing an immediate collision threat to specific segments. All obstacle parameters are pre-extracted into arrays upon initialization, allowing for vectorized operations instead of slow iterative loops.

The problem is solved using the Dense Normal Cholesky linear solver via `pyceres`. Due to Python limitations (), performance drops when increasing the number of threads, so the optimization is strictly restricted to a single thread. To maintain a predictable control rate, the solver is capped at a strict maximum number of iterations.

Latency Compensation

The implementation uses a warm start approach to maintain temporal coherence. However, because non-linear optimization introduces computational latency, the UGV physically moves while the solver runs. If the trajectory state was preserved blindly, the initial poses would be outdated by the time the next frame begins.

To resolve this, a latency compensation mechanism is implemented. Before the next optimization step begins, the system queries the most recent odometry estimate (filtered through EKF). The displacement ($\Delta x, \Delta y, \Delta\theta$) that accumulated during the

optimization is used to transform the trajectory matrix. This ensures the warm-started trajectory matches the robot's current pose before being passed to the solver. After these adjustments, the first pose in the trajectory is translated into linear (v) and angular (ω) velocities and published to the UGV's motor controllers.

4.4 Hardware

For the task Ubuntu 22.04 and ROS2 Jazzy were used.

Chapter 5

Experiments and Results

Chapter 6

Conclusions

Bibliography

[1] Sameer Agarwal, Keir Mierle, and The Ceres Solver Contributors. *Ceres Solver*. 2022. URL: http://ceres-solver.org.

[2] Kai O. Arras, Oscar Martinez Mozos, and Wolfram Burgard. "Using Boosted Features for the Detection of People in 2D Range Data". In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2007, pp. 3402–3407. DOI: 10.1109/ROBOT.2007.363998.

[3] Mona Forsman et al. "Bias of cylinder diameter estimation from ground-based laser scanners with different beam widths: A simulation study". In: *ISPRS Journal of Photogrammetry and Remote Sensing* 135 (2018), pp. 84–92. DOI: 10.1016/j.isprsjprs.2017.11.013.

[4] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. "A Formal Basis for the Heuristic Determination of Minimum Cost Paths". In: *IEEE Transactions on Systems Science and Cybernetics* 4.2 (1968), pp. 100–107. DOI: 10.1109/TSSC.1968.300136.

[5] Michael Hentschel and Bernardo Wagner. "Autonomous Robot Navigation Based on OpenStreetMap Geodata". In: *Proceedings of the IEEE Conference on Emerging Technologies and Factory Automation (ETFA)*. 2010, pp. 1–4. DOI: 10.1109/ETFA.2010.5625092.

[6] Anthony Mandow et al. "Experimental Kinematics for Wheeled Skid-Steer Mobile Robots". In: *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2007, pp. 1222–1227. DOI: 10.1109/IROS.2007.4399139.

[7] Javier Minguez, Florent Lamiraux, and Jean-Paul Laumond. "Motion Planning and Obstacle Avoidance". In: *Springer Handbook of Robotics*. Springer, 2008, pp. 827–852. DOI: 10.1007/978-3-540-30301-5_36.

[8] Thomas Moore and Daniel Stouch. "A Generalized Extended Kalman Filter Implementation for the Robot Operating System". In: *Proceedings of the 13th International Conference on Intelligent Autonomous Systems (IAS)*. Springer, 2015, pp. 335–348. DOI: 10.1007/978-3-319-08338-4_25.

[9] Viet Nguyen et al. "A Comparison of Line Extraction Algorithms Using 2D Laser Rangefinder for Indoor Mobile Robotics". In: *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2005, pp. 1929–1934. DOI: 10.1109/IROS.2005.1545234.

[10] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. 4th ed. Pearson, 2020.

[11] Christoph Rösmann et al. "Efficient Trajectory Optimization Using a Sparse Model". In: *Proceedings of the European Conference on Mobile Robots (ECMR)*. 2013, pp. 138–143.

[12] Christoph Rösmann et al. "Trajectory Modification Considering Dynamic Constraints of Autonomous Robots". In: *Proceedings of the 7th German Conference on Robotics (ROBOTIK)*. 2012, pp. 1–6.

[13] Fabian Schmidt et al. "Visual-Inertial SLAM for Unstructured Outdoor Environments: Benchmarking the Benefits and Computational Costs of Loop Closing". In: *Journal of Field Robotics* (2025). DOI: 10.1002/rob.22581.

[14] João Xavier et al. "Fast Line, Arc/Circle and Leg Detection from Laser Scan Data in a Player Driver". In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2005, pp. 3930–3935. DOI: 10.1109/ROBOT.2005.1570721.